Spike Extension Report

COS30002 | Artificial Intelligence for Game

Pattarapol Tantechasa, 103883220

Introduction

Extensions were completed across seven spikes:

- Graph Search (Task 4)
- Navigation with Graphs (Task 6)
- Tactical Analysis with PlanetWars (Task 10)
- Tactical Steering Hide (Task 13)
- Emergent Group Behaviour (Task 14)
- Agent Marksmanship (Task 15)
- Soldier on Patrol (Task 16)

Totalling nineteen extensions across topics ranging from graph search and pathfinding to steering behaviours, tactical decision-making, and finite state machines.

Note - Task 8 Spike GOAP: No extension instructions were provided for this task, so no extension work was undertaken. Only the baseline GOAP wizard-journey simulation was submitted. Its absence from this report is intentional.

Unit Learning Outcomes

ULO	Description
ULO1	Discuss and implement software development techniques to support the creation of AI behaviour in games, focusing on non-obvious problem-solving approaches.
ULO2	Understand and utilise a variety of graph and path-planning techniques.
ULO3	Design and create realistic movements for agents using steering force models to simulate natural and responsive behaviour.
ULO4	Design and develop agents capable of planning actions that require adaptive problem-solving techniques and innovative solutions.
ULO5	Combine multiple AI techniques to develop sophisticated game AI capable of solving intricate, multi-layered problems that arise in dynamic game environments.

1. Spike Extension Alignment Matrix

Spike	Extension	ULO1	ULO2	ULO3	ULO4	ULO5
Task 4 – Graph Search	Ext: Search Tree Visualisation	X	X			
Task 6 – Navigation with Graphs	Ext: Collision & Cost-Based Avoidance	X	X		X	X
Task 10 – PlanetWars	Ext 1: Asymmetric Maps	X			X	X
	Ext 2: Fog of War (FogBot)	X			X	X
Task 13 – Tactical Steering Hide	Ext 1: Obstacle Avoidance	X		X		
	Ext 2: Multiple Hunters	X			X	X
	Ext 3: Danger-Aware Path Scoring	X			X	X
	Ext 4: Hunter Chase, Eat & Respawn			X	X	X
Task 14 – Emergent Group Behaviour	Ext 1: Non-Overlapping Agents	X		X		
	Ext 2: Predator Avoidance			X	X	X
	Ext 3: Wall Avoidance	X		X		
	Ext 4: Factional Behaviours	X			X	X
Task 15 – Agent Marksmanship	Ext 1: Dynamic Range / CQC Shotgun	X		X	X	
	Ext 2: AoE Detonation & Proximity Targeting	X			X	
	Ext 3: Evasive Steering & Cover Seeking			X	X	X
Task 16 – Soldier on Patrol	Ext 1: Enemy Shoots Back	X			X	X
	Ext 2: FleeState & HealingState			X	X	X
	Ext 3: Ammo & Reload System	X			X	
	Ext 4: Multiple Agents	X			X	X

2. Justification for Each Extension

2.1 Task 4 – Spike Graph Search (Assignment 2)

Extension: Search Tree Visualisation

Two functions were added to the `GameNode` class: `render_board()` converts a flat board list into a 3×3 ASCII grid, and `print_path()` walks the parent chain from the winning node back to the root, then prints each intermediate board state with its depth level and move index.

- **ULO2** - The parent chain is the game tree's path structure; tracing it requires understanding how graph traversal accumulates state through node-to-node links.
- **ULO1** - Encoding the search path implicitly inside each node (via a parent pointer) rather than maintaining a separate external record is a non-obvious software design decision that enables path reconstruction without re-running the search.

2.2 Task 6 – Spike Navigation with Graphs (Assignment 3)

Extension: Agent Collision & Cost-Based Avoidance

When two agents collide, the losing agent respawns at a random location and that tile's edge cost is spiked by +20 units in the A* graph. The cost decays at 2.0 units/second over ~10 seconds (with a visual colour fade from red back to normal). Because A* always selects minimum-cost paths, agents automatically reroute around high-cost tiles - no explicit "avoid this tile" instruction is needed.

- **ULO2** - A* is applied to a dynamically modified graph, requiring an understanding of how edge cost changes directly affect path optimality guarantees.
- **ULO1** - Treating hazards as temporary cost increases rather than hard obstacles is a non-obvious insight: the same A* algorithm handles both static terrain costs and dynamic threats without any additional logic.
- **ULO4** - Agents independently replan routes in response to a changing cost landscape, adapting without any global coordination.
- **ULO5** - The system combines A* pathfinding, per-frame collision detection, dynamic graph cost modification, and time-based threat decay into one coherent adaptive navigation system.

2.3 Task 10 – Spike Tactical Analysis with PlanetWars (Assignment 5)

Extension 1: Asymmetric Maps

A custom map (`asym001`) was created with a deliberate 3× production asymmetry - Player 1's nearby planets have growth 4, 5, 5 while Player 2's have growth 1, 1, 1. TacticalBot achieved a 100% win rate from both the advantaged and disadvantaged side (20 runs each).

- **ULO1** - Using controlled map asymmetry as an empirical test is a non-obvious evaluation technique: it isolates whether the bot's scoring function compensates for positional disadvantage automatically, without being designed for it.
- **ULO4** - TacticalBot's scoring function (`40 × growth − 1 × distance`) naturally gravitates toward high-growth planets regardless of starting position - adaptive prioritisation not explicitly programmed for asymmetric conditions.
- **ULO5** - The multi-criteria decision system (garrison discipline, affordability checks, growth-weighted scoring) produces robust strategic behaviour in conditions it was never designed to handle.

Extension 2: Fog of War (FogBot)

FogBot extends TacticalBot with two behaviours: (1) stale-data compensation adjusts ship estimates by `estimated_ships = target.ships + target.growth × vision_age` to account for enemy growth during unobserved periods; (2) a scouting system sends a 3-ship fleet to the highest-priority unseen planet (ranked by `growth × vision_age`) once per tick to refresh intelligence.

- **ULO1** - Treating information age as a quantitative factor in an attack decision is non-obvious; it requires recognising that data reliability is itself a variable the AI must reason about.
- **ULO4** - FogBot plans under genuine uncertainty: it estimates enemy strength from stale data and allocates ships to scouting as well as attacking - adaptive resource allocation under incomplete information.
- **ULO5** - The agent integrates tactical targeting, per-planet vision tracking, stale-data estimation, and proactive scouting into a single unified decision-making system.

2.4 Task 13 – Tactical Steering Hide (Portfolio Task 13)

Extension 1: Obstacle Avoidance

A proximity-based repulsion force was added that runs every frame for every agent. It computes each agent's overlap with each obstacle's danger zone and applies a proportional push force away from the obstacle centre. A hard position correction also ejects any agent that fully penetrates an obstacle, preventing tunnelling at high speed.

- **ULO1** - Blending a repulsion force on top of any active primary steering (rather than switching modes) is a non-obvious architectural choice that keeps avoidance seamless and non-disruptive.
- **ULO3** - Agents curve naturally around obstacle edges during transit instead of clipping through them, producing more realistic movement through the environment.

Extension 2: Multiple Hunters

The simulation was extended from one hunter to three independent hunters (red, orange, pink), each with its own wander state. Each frame, the prey identifies the nearest hunter as the primary threat and recalculates all hide spots relative to that hunter.

- **ULO1** - Selecting the nearest threat per-frame (rather than tracking a fixed target) is a non-obvious design that keeps the prey reactive without requiring any communication between hunters.
- **ULO4** - The prey must adaptively switch its reference threat as hunters move, making real-time planning decisions under spatial uncertainty.
- **ULO5** - Hunters independently transition between wander and chase, creating complex and shifting threat pressure on the prey with no coordinated logic between them.

Extension 3: Danger-Aware Path Scoring

A `path_exposure()` method was added to the prey. For each candidate hiding spot, it projects each hunter's position onto the straight-line segment from the prey to that spot, and accumulates exposure if any hunter falls within 150px of the path. The final selection score is `dist_to_spot + path_exposure × 2.0`.

- **ULO1** - The key insight - that the travel route carries risk, not just the destination - is a non-obvious problem-solving contribution. The geometric projection technique captures route danger without any pathfinding algorithm.
- **ULO4** - The prey now plans ahead: it will accept a longer route to a hiding spot if the approach path is safer, rather than always picking the nearest option.
- **ULO5** - The behaviour combines geometric analysis, steering, and multi-threat path evaluation into a single real-time decision each frame.

Extension 4: Hunter Chase, Eat & Respawn

Each hunter gained a two-state behaviour: wander when line-of-sight to the prey is blocked, chase when it is clear. Line-of-sight is tested each frame using the quadratic segment-circle intersection formula against all obstacles. When chasing, the hunter uses a pursuit behaviour that predicts the prey's future position. If any hunter closes within 25px, the prey teleports to a random position and a catch counter increments.

- **ULO3** - Pursuit steering produces a realistic intercept arc - the hunter leads the prey rather than following it - creating a believable predator-prey chase.
- **ULO4** - The wander→chase state transition is driven by environmental sensing (line-of-sight), constituting adaptive decision-making based on what the hunter can currently observe.
- **ULO5** - The simulation combines hide spot geometry, obstacle avoidance, multi-threat path scoring, line-of-sight raycasting, and pursuit steering into a self-sustaining predator-prey game loop.

2.5 Task 14 – Emergent Group Behaviour (Portfolio Task 14)

Extension 1: Non-Overlapping Agents

A rigid body collision resolution pass runs after the steering physics update each frame. For every agent pair whose distance is less than the sum of their radii, both agents are pushed apart equally along the separation axis. This is a hard position constraint - it always runs regardless of steering weight settings.

- **ULO1** - Recognising the distinction between a soft steering force and a hard position constraint, and applying both in sequence without interference, is a non-obvious architectural design decision.
- **ULO3** - Under high cohesion settings, agents now form compact clusters with visible gaps between individuals rather than invisibly stacking on the same pixel - a direct improvement in physical realism.

Extension 2: Predator Avoidance

Three red predator agents were added; each independently seeks the nearest prey agent every frame at slightly higher speed. Prey agents gained a `flee_predator()` force that activates when any predator enters a 160px panic radius, producing a push force weighted by inverse distance.

- **ULO3** - The flee force integrates cleanly with the existing weighted-sum framework, producing responsive, distance-sensitive evasion movement.
- **ULO4** - The flock fractures and reforms dynamically based on predator proximity - an adaptive group response with no scripted rules governing the behaviour.
- **ULO5** - Emergent panic arises from just three forces interacting: flee pulls individuals away, cohesion pulls them back, separation prevents crowding - producing realistic collective behaviour from simple components.

Extension 3: Wall Avoidance

Each frame, every agent projects three feeler rays from its position: one straight ahead (120px), one 35° left (78px), and one 35° right (78px). Each ray is tested against a static horizontal wall segment using the 2D parametric line intersection formula. When an intersection is found, a velocity-space steering force is applied away from the wall normal, scaled by the proximity of the hit. A secondary hard ejection pass prevents high-speed tunnelling.

- **ULO1** - Feeler raycasting is a non-obvious technique: it requires no global map or pathfinding - agents reason only about what their projected rays immediately encounter.
- **ULO3** - Agents curve naturally around wall ends without stopping or teleporting, demonstrating fluid environmentally-aware steering.

Extension 4: Factional Behaviours

Three distinct agent factions were created: Prey (20 cyan agents - flock + flee predators), Predators (3 red agents - seek nearest prey, no flocking), and Scavengers (7 yellow agents - flock among themselves + `follow_predators()` force that trails predators at a safe distance). A faction integer on each agent filters the neighbour detection so cohesion, separation, and alignment only consider same-species neighbours.

- **ULO1** - Extending the weighted-sum framework to multi-species dynamics using a faction integer filter is a non-obvious design that requires no structural changes to the core flocking logic.
- **ULO4** - Each faction exhibits an implicit adaptive policy: predators hunt, prey scatter and regroup, scavengers trail opportunistically - all from per-agent local rules.
- **ULO5** - A coherent three-level food chain ecosystem emerges with no scripted coordination between species, demonstrating that complex multi-agent dynamics can arise entirely from simple local rules.

2.6 Task 15 – Agent Marksmanship (Portfolio Task 15)

Extension 1: Dynamic Range Finding & Close Quarters Combat (Shotgun)

A Shotgun weapon was added with a 150px effective range, 20° spread cone, and 8 pellets per shot. To use it, the `AttackerAgent` was given movement physics (velocity, mass, max speed). When the Shotgun is active and the target is beyond 150px, the attacker applies a seek force to close the distance; once within range, it brakes to a stop and fires.

- **ULO1** - Range-gated mode switching (approach vs. engage) transforms effective range from a passive weapon stat into an active AI decision variable - a non-obvious design.
- **ULO3** - The seek-then-brake movement reuses arrive-style physics from prior spikes, showing how the same steering model generalises naturally to a combat context.
- **ULO4** - The attacker must evaluate its own spatial position relative to the target before choosing whether to move or fire, constituting adaptive positional decision-making.

Extension 2: AoE Detonation & Proximity Targeting

The Hand Grenade was modified to fire an `ExplosiveProjectile` that travels to a fixed aim point and detonates on arrival, checking if the target is within an 80px blast radius. The aim point is shifted 30px below the predicted body intercept (`aim_point = predicted_body_pos + Vector2D(0, -PROXIMITY_OFFSET)`), deliberately targeting the area around the target's feet rather than its body centre.

- **ULO1** - Deliberately aiming at the blast area rather than the target body is a non-obvious design decision - the attacker reasons about coverage radius, not point precision.
- **ULO4** - Proximity offset targeting is an adaptive trade-off: for a slow, inaccurate explosive weapon, guaranteeing the detonation lands close is more effective than attempting a direct body hit.

Extension 3: Evasive Steering & Cover Seeking Logic

The `TargetAgent` was given a three-state FSM: PATROL (normal waypoint seeking), EVADE, and SEEK_COVER. Each frame, `detect_threat()` flags projectiles that are slow (below 200px/s) and within 200px. On a detected threat: if the nearest obstacle is within 220px, the target enters SEEK_COVER and uses `arrive()` to move behind the obstacle relative to the attacker's position; if no obstacle is close enough, it enters EVADE and applies a `flee()` force away from the incoming projectile.

- **ULO3** - Both active states use steering forces (`flee()` in EVADE, `arrive()` in SEEK_COVER), demonstrating steering behaviours applied reactively in a tactical context.

- **ULO4** - The target evaluates two independent spatial conditions - threat speed and obstacle proximity - and selects the appropriate adaptive response each frame.
- **ULO5** - The extension integrates FSM decision logic, threat detection, spatial cover geometry, and two distinct steering behaviours into a unified reactive system.

2.7 Task 16 – Soldier on Patrol (Portfolio Task 16)

Extension 1: Enemy Shoots Back

Each enemy gained an `update()` method that fires a projectile toward the nearest soldier every 2 seconds when a soldier is within 250px. A health system (`health`, `max_health`, `take_damage()`) was added to the `Agent` class. The existing `Projectile` class was updated to carry a damage parameter so soldiers and enemies share the same projectile type with asymmetric damage values (soldiers deal 25, enemies deal 10).

- **ULO1** - Adding a damage parameter to an existing shared class rather than creating a separate enemy projectile type is a non-obvious scalable design decision.
- **ULO4** - Enemies independently monitor proximity and fire when the condition is met - a reactive AI subsystem that produces genuine bidirectional combat.
- **ULO5** - The simulation now involves soldiers patrolling, detecting threats, attacking enemies, and taking return fire - a multi-agent interaction with emergent combat dynamics.

Extension 2: FleeState & HealingState

Two new FSM states were added. `FleeState` activates when health drops below 30hp and an enemy is in range - the soldier applies `flee()` away from the nearest enemy, then transitions to `HealingState` when the threat clears. `HealingState` uses `arrive()` to navigate to a fixed `HealingStation` and recovers 20hp/second within 20px of it. A hysteresis gap is applied: soldiers enter low-health states below 30hp but only exit above 80hp, preventing rapid oscillation between states.

- **ULO3** - Both states are driven by steering forces (`flee()` and `arrive()`), demonstrating how steering behaviours integrate naturally into a formal FSM.
- **ULO4** - The hysteresis gap (30hp entry / 80hp exit) is an adaptive design technique that solves the state-oscillation problem - without it, a soldier taking chip damage at the boundary would flip states every frame despite being technically correct.
- **ULO5** - The four-state FSM with priority Flee > Attack > Heal > Patrol combines health monitoring, spatial navigation, steering forces, and multi-condition transitions into a cohesive multi-layered decision system.

Extension 3: Ammo & Reload System

`AttackState` was given a 6-round clip that fires once per second. When the clip empties, a 2-second reload phase begins during which no shots fire; on completion, the clip refills automatically. All ammo state (`ammo`, `reloading`, `reload_timer`) is stored on the state object itself, not on the agent.

- **ULO1** - Storing resource state inside the FSM state object rather than on the agent is the correct application of the State pattern - a non-obvious design decision that keeps ammo logic cleanly isolated to the one state where it is relevant.
- **ULO4** - The reload constraint makes the agent implicitly resource-aware: it must wait before acting again, adding tactical nuance to the combat behaviour without any explicit planning logic.

Extension 4: Multiple Agents

Three soldiers were instantiated simultaneously, each with an independently randomised patrol path and its own FSM instance. Because all FSM state (ammo, reload timer, health) is held inside the agent or its state objects rather than shared global variables, no architectural changes were needed to support three agents. The HUD displays all three soldiers' state, ammo, and health in real time.

- **ULO1** - Clean instance isolation across three agents required no refactoring - this is the direct payoff of the State pattern and OO design decisions made in earlier extensions.
- **ULO4** - Each soldier independently evaluates its own threat landscape and health, producing per-agent adaptive behaviour rather than a uniform group response.
- **ULO5** - Three concurrent agents each running a full four-state FSM with health, ammo, and steering behaviours form a multi-layered, multi-agent system that could not arise from any single AI technique alone.

Conclusion

Across seven spikes and nineteen extensions, this work demonstrates consistent and broad engagement with all five ULOs.

ULO1 is present in every spike. Each extension was built around a non-obvious software design decision: from parent-chain tracing in Task 4, to cost-function modification in Task 6, to State-pattern resource encapsulation in Task 16. The common thread is choosing the right architectural approach over the obvious one.

ULO2 is most directly addressed by the assignment spikes. Task 4's search tree visualisation demonstrates a working understanding of game-state graph structure and path tracing. Task 6's dynamic cost avoidance shows A* applied to a graph that changes at runtime - extending beyond standard static pathfinding.

ULO3 is built up progressively through Tasks 13, 14, and 15. Steering force models are applied in increasingly complex contexts: individual tactical hiding, multi-agent flocking, and reactive combat evasion - demonstrating that the same physical steering model scales naturally across very different problem domains.

ULO4 is demonstrated wherever agents evaluate conditions and select responses adaptively - from FogBot estimating enemy strength under incomplete information (Task 10), to the prey choosing safer routes over shorter ones (Task 13), to the hysteresis-gated FSM transitions that prevent state instability (Task 16).

ULO5 is demonstrated by the most advanced extension in each spike. The predator-prey game loop in Task 13, the three-faction ecosystem in Task 14, and the four-state combat FSM in Task 16 all produce behaviours that cannot be attributed to any single technique - they are the result of multiple AI concepts composed together deliberately. Complexity in each case was achieved by combining simple, well-understood mechanisms rather than by introducing artificial complexity.